Eventtz — Architecture Guide

A practical map of where things live, how to read the code, and a one-week path to understand the product end to end.

Canonical reference in the repo: `cursor.md`

How to use this doc for learning

1. Skim §§1–5 for the map (layers, golden rules, how to trace).

2. Follow **§8 Week of deep reading** day by day — one feature at a time.

3. For each feature, use the **Trace method** (UI → submit → `lib/*Api` → `/api/v1/…` → `http/` → logic → DB). Do not skip layers.

4. Treat **§6.1 Auth** as the template walkthrough: every other feature in §6 should eventually get the same UI→backend depth as you go.

When you finish a day's feature, tick the checklist in §8 and note open questions. We will extend §6 walkthroughs feature by feature until the whole product is covered at that depth.

---

1. Big picture

Eventtz is a monorepo with two apps:

- **backend/** — Python (Poetry), FastAPI, Supabase (Postgres + Auth + Storage)
- **frontend/** — Next.js (App Router), TypeScript, Tailwind, Axios

Request flow

```
Browser → frontend/app/page.tsx (thin route)
       → frontend/features/* (screens, hooks)
       → frontend/lib/*Api.ts (HTTP)
       → backend/app/features/*/http/*.py (routes)
       → backend/app/features/*/*.py (business logic)
       → Supabase / Stripe / OpenAI
```

Golden rules

Rule	Backend	Frontend
Dependency direction	`http/` → logic → `core/` (never logic → http)	`app/` → `features/` → `lib/` (never axios in page.tsx)
One registration point	All routes in `app/http/router.py`	Routes only compose feature views
API shapes	`app/contracts/`	`lib/api-types.ts`, `lib/domain-types.ts`
Feature grouping	`app/features/<name>/`	`features/<name>/` + matching `lib/*Api.ts`

---

2. Repository layout

```
eventtz/
├── AGENTS.md              # Tells Cursor agents to follow cursor.md
├── cursor.md              # Full architecture doc (keep in sync)
├── docs/
│   └── eventtz-architecture-guide.md   # This document
├── backend/
│   ├── app/
│   │   ├── main.py        # FastAPI app, CORS, /health, mounts /api/v1
│   │   ├── core/          # config, db, logging, errors
│   │   ├── contracts/     # Pydantic request/response models
│   │   ├── http/router.py # ONLY place routers are registered
│   │   ├── features/      # Product areas (auth, bookings, chat, …)
│   │   └── jobs/          # CLI cron (e.g. booking maintenance)
│   ├── sql/               # Numbered Postgres migrations (001…036)
│   └── tests/             # pytest
└── frontend/
    ├── app/               # Next.js routes (thin shells)
    ├── features/          # Screens, hooks, view models per product area
    ├── components/        # Shared UI, auth, portal/admin shells
    └── lib/               # axios + *Api.ts HTTP clients
```

---

3. Backend architecture

Layers

Area	Path	Responsibility
App shell	`app/main.py`	FastAPI, CORS, mounts router at `/api/v1`, `GET /health`
Router aggregator	`app/http/router.py`	**Only** place that `include_router`s feature HTTP modules
Core	`app/core/`	`config`, `logging`, `constants`, `startup`, `errors`, `db`
Contracts	`app/contracts/`	Pydantic request/response models; shared literals in `types.py`
Features	`app/features/<name>/`	One product area per folder
Jobs	`app/jobs/`	One-shot CLI scripts for Railway cron

Typical feature folder shape

```
features/<name>/
├── http/       # FastAPI routers (thin: auth → logic → contract)
├── *.py        # Business logic modules
└── db*.py      # Table access for this feature
```

Rule: Feature logic must **not** import from feature `http/`.

Core files (`backend/app/core/`)

File	Purpose
`config.py`	All env vars (`Settings`) — Stripe, Supabase, OpenAI, booking fees
`db.py`	Supabase client, `get_db()`, `rows()`, `one_row()`
`errors.py`	`AppError` + global HTTP error handler
`logging.py`	Structured logging
`constants.py`	Cookie names (`eventtz_access_token`, etc.)
`startup.py`	App startup hooks

Contracts (`backend/app/contracts/`)

Read these to learn what an endpoint accepts/returns.

File	Domain

`types.py`	`UserType`, shared enums
`booking.py`	Booking list/detail/create payloads
`chat.py`	Conversations, messages
`disputes.py`	Problem reports
`payments.py`	Checkout, completion confirm
`vendor.py`	Vendor profile, explore
`admin.py`	Admin console DTOs
`notifications.py`	Notification feed items
`reviews.py`	Booking reviews
`geo_address.py`	UK address lookup

Router registration (`backend/app/http/router.py`)

Every API lives under `/api/v1`. Registration order:

1. auth
2. chat
3. user_settings
4. vendor_profile
5. vendor_onboarding_ai
6. vendor_bookings
7. admin_vendors
8. admin_platform
9. vendors_explore
10. client_booking_requests
11. client_geo
12. client_notifications
13. vendor_notifications

14. vendor_payments
15. media
16. realtime
17. stripe_webhooks

CLI jobs (`backend/app/jobs/`)

Module	Purpose
`run_booking_maintenance.py`	Hourly: post-event completion reminders + 48h payout auto-release

Run: `PYTHONPATH=. python -m app.jobs.run_booking_maintenance` (from `backend/`)

---

4. Frontend architecture

Layers

Layer	Path	Responsibility
Routes	`app/**/page.tsx` , `layout.tsx`	Thin shells: auth gates, compose one feature view
Features	`features/<domain>/**`	Screens, hooks, view models owned by that flow
Shared UI	`components/ui/**`	Reusable primitives (Modal, inputs, etc.)
Cross-cutting UI	`components/auth` , `*-shell`	Auth context, portal/admin chrome
Domain wizard	`components/vendor-onboarding/**`	Multi-step vendor profile
API & types	`lib/**`	`*Api.ts` modules, axios, error helpers

Rules:

- All backend calls go through `lib/` *(typically* `Api.ts`), not from `app/` route files.
- Tailwind for all styling.

Route groups (`frontend/app/`)

Prefix	Who	Shell
`/`	Public	Landing
`/explore`	Public	Marketplace search
`/login`, `/register`	Public	Unified auth entry
`/client/*`	Logged-in clients	`PortalShell`
`/vendor/*`	Logged-in vendors	`PortalShell`
`/admin/*`	Admins	`AdminShell`
`/compliances/*`	Public	Legal pages

Client navigation (`components/portal-shell/portalNav.ts`)

Dashboard → Browse → Favorites → Bookings → Messages → Disputes → Settings

Vendor navigation

Dashboard → Bookings → Messages → Profile → Payments → Disputes → Settings

Lib modules (`frontend/lib/`)

File	Role
`axios.ts`	Shared axios instance + base URL
`auth-interceptors.ts`	401 → refresh + retry
`auth-api.ts`	Sign in/up/out, refresh, `/me`
`api-errors.ts`	`getApiErrorDetail`, error parsing
`api-types.ts`	Shared API-related TS types
`domain-types.ts`	Domain enums / literals
`clientBookingsApi.ts`	Client booking list/detail/actions
`vendorBookingsApi.ts`	Vendor booking list/detail/actions
`bookingCheckoutApi.ts`	Stripe Checkout session
`chatApi.ts`	Conversations and messages

`clientNotificationsApi.ts`	Client notification feed
`vendorNotificationsApi.ts`	Vendor notification feed
`bookingDisputesApi.ts`	Participant disputes
`vendorPaymentsApi.ts`	Stripe Connect onboarding
`vendorProfileApi.ts`	Vendor profile CRUD
`clientExploreApi.ts`	Marketplace / browse
`addressApi.ts`	UK address autocomplete
`mediaApi.ts`	Image upload URLs
`adminPlatformApi.ts`	Admin dashboard, bookings, disputes
`adminVendorsApi.ts`	Admin vendor management
`userSettingsApi.ts`	User settings
`clientOnboardingApi.ts`	Client first-visit onboarding
`reviewsApi.ts`	Booking reviews
`realtimeBus.ts` / `realtimeHooks.ts`	SSE-driven UI refresh

5. How to trace any bug

1. Start at the **URL** in the browser (e.g. `/client/bookings`).

2. Find **`app/.../page.tsx`** — see which `features/*` view it imports.

3. Open that **view** — find which `lib/*Api.ts` it calls.

4. **Grep the API path** in backend (e.g. `booking-requests`) → lands in `features/bookings/http/client.py`.

5. **Route file** calls business logic (`status.py`, `payments.py`, etc.).

6. Read **`contracts/booking.py`** for request/response shapes.

6. Feature-by-feature breakdown

Each subsection starts with a short map (files + endpoints). Features marked **Walkthrough** include the full UI → API → backend path. **Auth is complete** — use it as the model when we deepen Bookings, Chat, Payments, etc. later this week.

The Trace method (do this for every feature)

```
1. Browser URL          → frontend/app/**/page.tsx (thin)
2. Feature view/hook    → frontend/features/... (form, onSubmit)
3. HTTP client          → frontend/lib/*Api.ts (exact path + body)
4. Axios + cookies      → frontend/lib/axios.ts (withCredentials)
5. Backend route        → backend/app/features/*/http/*.py
6. Guarding             → features/auth/http/guards.py (if authenticated)
7. Business logic       → features/*/*.py (never import http/)
8. Contracts            → backend/app/contracts/*.py
9. Database             → core/db.py + feature queries / sql migrations
```

Always open the files in that order. Sketch the sequence for yourself before reading the next feature.

6.1 Auth — Walkthrough (sign up, sign in, session, roles)

What it does: Supabase Auth (or local mode) behind FastAPI. Browser stores **HttpOnly cookies** (`eventtz_access_token`, `eventtz_refresh_token`), never tokens in `localStorage`. `/me` hydrates the user from Postgres. Portal layouts gate by role.

Product URLs

URL	Purpose	
`/register?type=client|	vendor`	Unified register
`/login`	Unified client/vendor login	
`/admin/login`	Admin-only login	
`/client/`, `/vendor/`, `/admin/*`	Portals (require session + role)	

Legacy `/client/register`, `/vendor/login`, etc. redirect into the unified pages via `AuthRouteRedirect`.

Step A — UI (register)

File	Role
`frontend/app/register/page.tsx`	Route shell → `UnifiedRegisterView`
`frontend/features/auth/UnifiedRegisterView.tsx`	Form: email, password, account type
`frontend/features/auth/useUnifiedRegister.ts`	`onSubmit`
`frontend/components/auth/AuthPageShell.tsx`	Shared auth chrome

Step B — On submit (register)

1. `useUnifiedRegister.onSubmit` calls `signUp(email, password, { userType })` from `useAuth()`.

2. `AuthProvider.signUp` (`components/auth/AuthProvider.tsx`) → `apiSignUp` in `lib/auth-api.ts`.

3. `POST /api/v1/auth/signup` with body `{ email, password, user_type: "client" | "vendor" }`.

4. On success, `refreshUser()` → `GET /api/v1/auth/me`.

5. If the API returned a “confirm email” style `message`, show it; else `router.push(resolvePostAuthPath(next, accountType))` (`features/auth/authRouting.ts`).

Default landings: client → `/client/dashboard`, vendor → `/vendor/dashboard`, admin → `/admin/dashboard`. `?next=` is honored only if safe for that role.

Step C — Login

File	Role
`frontend/app/login/page.tsx`	→ `UnifiedLoginView`
`features/auth/useUnifiedLogin.ts`	`signIn` → `getMe` → redirect
`features/admin/auth/useAdminLogin.ts`	Same sign-in; rejects non-admin via `signOut`

Step D — Frontend HTTP + session plumbing

File	Role
`lib/auth-api.ts`	`signUp`, `signIn`, `signOut`, `refreshSession`, `getMe`
`lib/axios.ts`	Shared client, `withCredentials: true`

`lib/auth-interceptors.ts`	On 401 → `refreshSession` + retry; installed by `AuthProvider`
`app/layout.tsx`	Wraps app in `<AuthProvider>`

Step E — Backend HTTP

Router: `backend/app/http/router.py` includes auth. Prefix `/auth` under `/api/v1`.

Method	Path	Handler
POST	`/api/v1/auth/signup`	`features/auth/http/routes.py` → `signup`
POST	`/api/v1/auth/signin`	→ `sign_in`
POST	`/api/v1/auth/signout`	→ `sign_out`
POST	`/api/v1/auth/refresh`	→ `refresh_session`
GET	`/api/v1/auth/me`	→ `me`

Thin routes call:

Module	Key symbols
`features/auth/supabase.py`	`SupabaseAuthService.sign_up`, `sign_in_with_password`, …
`features/auth/local_store.py`	Local-dev register/authenticate (when enabled)
`features/auth/session.py`	`set_session_cookies`, `clear_session_cookies`, `get_current_user_or_raise`
`features/auth/accounts.py`	`hydrate_user_from_db`, `upsert_user_profile`
`features/email/dispatch.py`	`send_welcome_email` after signup

Cookies (`backend/app/core/constants.py`): `eventtz_access_token` (~1h), `eventtz_refresh_token` (~14d). HttpOnly; Secure/SameSite depend on env. Frontend never reads them — browser attaches via cookies on same-origin `/api/v1` (Next rewrite in `next.config.ts`).

Step F — Guarding portals

Frontend layouts

Layout	Chain
`app/client/(portal)/layout.tsx`	`RequireAuth` → `RequireClient`

`app/vendor/(portal)/layout.tsx`	`RequireAuth` → `RequireVendor` → often `RequireVendorApproved`
`app/admin/(console)/layout.tsx`	`RequireAuth` → `RequireAdmin`

Guards live in `frontend/components/auth/Require*.tsx`.

Backend API (every protected feature route): `features/auth/http/guards.py` — `require_client`, `require_vendor`, `require_admin`, `require_super_admin`.

Auth sequence diagrams

Register

```
/register
  → UnifiedRegisterView / useUnifiedRegister.onSubmit
  → AuthProvider.signUp → auth-api.signUp
  → POST /api/v1/auth/signup
  → local_store OR SupabaseAuthService.sign_up
  → upsert_user_profile (+ welcome email)
  → Set-Cookie access + refresh
  → GET /api/v1/auth/me
  → resolvePostAuthPath → portal dashboard
  → layout RequireAuth + role guard
```

Login

```
/login
  → useUnifiedLogin.onSubmit
  → POST /api/v1/auth/signin → cookies
  → refreshUser + getMe
  → resolvePostAuthPath(next, user_type)
```

Auth file index

```
Frontend: app/register, app/login, app/admin/login,
  features/auth/*, features/admin/auth/*,
  components/auth/*, lib/auth-api.ts, lib/auth-interceptors.ts, lib/axios.ts
Backend: features/auth/http/{routes,guards,dependencies}.py,
  session.py, accounts.py, supabase.py, local_store.py,
  core/constants.py, http/router.py
```

Your study task for Auth: click Register in the UI, watch Network for `/api/v1/auth/signup`, then open `routes.py` and walk into `supabase.py` / `session.py`. Repeat for Sign in and for

visiting `/client/dashboard` while logged out (watch the redirect).

6.2 Client bookings — create, list, cancel, pay, complete

(Map only for now — deepen with the Trace method on Days 3–4)

Backend

File	Purpose
`features/bookings/http/client.py`	All client booking HTTP routes
`features/bookings/create.py`	Create booking / vendor quote
`features/bookings/status.py`	Accept, decline, cancel
`features/bookings/payments.py`	Checkout, refunds, completion, auto-release
`features/bookings/pricing.py`	Server-side pricing breakdown
`features/bookings/pricing_refresh.py`	Recompute before checkout
`features/bookings/queries.py`	List/detail queries
`features/bookings/completion_rules.py`	48h auto-release timing
`features/bookings/venue.py`	Event address updates
`features/bookings/reviews.py`	Post-event reviews
`contracts/booking.py`, `payments.py`	API shapes

Key endpoints

Endpoint	Purpose
`GET /client/booking-requests`	List (active/completed/closed)
`GET /client/booking-requests/{id}`	Detail
`POST /client/booking-requests`	Create from browse
`PATCH /client/booking-requests/{id}/status`	Accept/decline quote
`POST /client/booking-requests/{id}/cancel`	Cancel
`POST /client/booking-requests/{id}/checkout`	Stripe Checkout URL
`POST /client/booking-requests/{id}/checkout/sync`	Post-redirect sync

`POST /client/booking-requests/{id}/confirm-completion`	Mark event done
`PATCH /client/booking-requests/{id}/venue`	Event address

Frontend

File	Purpose
`app/client/(portal)/bookings/page.tsx`	Bookings route
`app/client/(portal)/bookings/[bookingId]/page.tsx`	Booking detail
`app/client/(portal)/bookings/[bookingId]/pay/page.tsx`	Pay redirect
`features/client/bookings/ClientBookingsView.tsx`	Main bookings UI
`features/client/bookings/ClientBookingsPortal.tsx`	Portal wrapper
`features/client/bookings/clientBookingViewModel.ts`	View model
`features/bookings/MasterDetailLayout.tsx`	List + detail layout
`features/bookings/BookingDetailPanel.tsx`	Detail panel
`features/bookings/BookingCompletionBanner.tsx`	Post-event banner
`features/bookings/bookingConfirmCopy.ts`	All booking copy/strings
`features/client/browse/VendorBookingModal.tsx`	Create booking from browse
`lib/clientBookingsApi.ts`	HTTP client
`lib/bookingCheckoutApi.ts`	Checkout HTTP

Vendor bookings — quotes, accept/decline, adjustments, complete

Backend

File	Purpose
`features/bookings/http/vendor.py`	Vendor booking HTTP routes
`features/bookings/adjustments.py`	Price adjustments
`features/bookings/calendar.py`	maxBookingsPerDay enforcement
`features/bookings/line_item_validation.py`	Quote line validation

Key endpoints

Endpoint	Purpose
`GET /vendor/booking-requests`	List
`GET /vendor/booking-requests/{id}`	Detail
`POST /vendor/booking-requests/vendor-quote`	Send quote
`PATCH /vendor/booking-requests/{id}/status`	Accept/decline/cancel
`PUT /vendor/booking-requests/{id}/adjustments`	Updated pricing
`POST /vendor/booking-requests/{id}/confirm-completion`	Mark event done

Frontend

File	Purpose
`app/vendor/(portal)/bookings/page.tsx`	Bookings route
`features/vendor/bookings/VendorBookingsView.tsx`	Main UI
`features/vendor/quotes/VendorQuoteFormModal.tsx`	Send quote
`features/vendor/bookings/VendorBookingResponseFlow.tsx`	Accept/decline flow
`lib/vendorBookingsApi.ts`	HTTP client

Payments (Stripe Connect)

Backend

File	Purpose
`features/payments/stripe.py`	Stripe SDK wrapper
`features/payments/http/vendor.py`	Connect onboarding
`features/payments/http/webhooks.py`	Stripe webhooks
`features/bookings/payments.py`	Booking money orchestration

Endpoints

Endpoint	Purpose
`POST /vendor/payments/connect-account`	Stripe Express onboarding

`GET /vendor/payments/status`	Charges/payouts enabled
`POST /webhooks/stripe`	Payment + account webhooks

Frontend

File	Purpose
`app/vendor/(portal)/payments/page.tsx`	Payments page
`features/vendor/payments/VendorPaymentsView.tsx`	Connect UI
`lib/vendorPaymentsApi.ts`	HTTP client
`components/vendor-onboarding/steps/StepPayment.tsx`	Onboarding Stripe step

Money model (summary)

- `booking_requests.status` — request lifecycle (pending/accepted/cancelled/completed)
- `booking_requests.payment_status` — money lifecycle (unpaid/paid/refunded/payout_released)
- Client pays via Stripe Checkout → funds held → vendor paid via explicit Transfer after mutual confirm or 48h auto-release

---

Chat / messaging

Backend

File	Purpose
`features/chat/http/routes.py`	HTTP routes
`features/chat/http/handlers.py`	Handler helpers
`features/chat/service.py`	Business logic
`features/chat/db.py`, `db_conversations.py`, `db_messages.py`	DB
`contracts/chat.py`	API shapes

Endpoints

Endpoint	Purpose
`GET /chat/unread-count`	Badge count
`GET /chat/conversations`	Inbox

`POST /chat/conversations`	Start thread
`GET /chat/conversations/{id}/messages`	History
`POST /chat/conversations/{id}/messages`	Send message

Frontend

File	Purpose
`app/client/(portal)/messages/page.tsx`	Client inbox
`app/vendor/(portal)/messages/page.tsx`	Vendor inbox
`features/chat/ChatPortalView.tsx`	Inbox + thread UI
`features/chat/ChatDrawer.tsx`	Drawer variant
`lib/chatApi.ts`	HTTP client
`features/realtime/*` via `lib/realtimeHooks.ts`	Live updates

Notifications (in-app)

Backend: `features/notifications/service.py`, `copy.py`, `http/client.py`, `http/vendor.py`

Frontend: `features/notifications/NotificationsPageView.tsx`, `lib/clientNotificationsApi.ts`, `lib/vendorNotificationsApi.ts`

Disputes (“Report a problem”)

Backend: `features/bookings/disputes.py`, `http/disputes_http.py`, `features/admin/dispute_ops.py`

Frontend: `features/bookings/BookingDisputeSection.tsx`, `features/disputes/ParticipantDisputesPortalView.tsx`, `lib/bookingDisputesApi.ts`, `features/admin/disputes/AdminDisputesView.tsx`

Reviews

Backend: `features/bookings/reviews.py`, `features/admin/review_ops.py`

Frontend: `features/reviews/BookingReviewPanel.tsx`, `features/client/bookings/ClientBookingReviewForm.tsx`, `lib/reviewsApi.ts`

Vendor profile & onboarding

Backend: `features/vendors/profile.py`, `payload_validation.py`, `moderation.py`, `list_pricing.py`, `onboarding_ai.py`, `http/profile.py`, `http/onboarding_ai.py`

Frontend: `components/vendor-onboarding/*` (wizard), `app/vendor/(portal)/profile/page.tsx`, `lib/vendorProfileApi.ts`, `lib/mediaApi.ts`

Marketplace / browse

Backend: `features/vendors/http/explore.py`, `search.py`, `search_ai.py`

Frontend: `app/explore/page.tsx`, `features/marketplace/MarketplaceExploreView.tsx`, `features/client/browse/`, *`lib/clientExploreApi.ts`*, *`features/landing/`*

Client onboarding (first visit)

Backend: `features/settings/client_onboarding.py`, migration `035_client_preferred_name.sql`

Frontend: `features/client/onboarding/*`, `lib/clientOnboardingApi.ts`

Settings & contact visibility

Backend: `features/settings/http/routes.py`, `features/settings/contact.py`

Frontend: `features/settings/SettingsView.tsx`, `lib/userSettingsApi.ts`

Geo / UK addresses

Backend: `features/geo/uk_address.py`, `os_places.py`, `getaddress.py`, `http/routes.py`

Frontend: `components/ui/AddressFinderInput.tsx`, `lib/addressApi.ts`

Media uploads

Backend: `features/media/upload.py`, `http/routes.py`

Frontend: `lib/mediaApi.ts`

Admin console

Backend

File	Purpose
`features/admin/http/platform.py`	Dashboard, bookings, disputes, audit, financials
`features/admin/http/vendors.py`	Vendor approval, insights
`features/admin/booking_ops.py`	Booking admin actions
`features/admin/dispute_ops.py`	Dispute resolution
`features/admin/review_ops.py`	Review moderation
`features/admin/audit.py`	Audit log
`features/admin/dashboard_queries.py`	Metrics

Frontend

File	Purpose
`components/admin-shell/AdminShell.tsx`	Admin chrome
`features/admin/dashboard/AdminDashboardView.tsx`	Dashboard
`features/admin/bookings/AdminBookingsView.tsx`	Bookings list
`features/admin/vendors/AdminVendorsView.tsx`	Vendors
`features/admin/disputes/AdminDisputesView.tsx`	Disputes
`features/admin/reviews/AdminReviewsView.tsx`	Reviews

`lib/adminPlatformApi.ts`	HTTP client

Admin routes: `/admin/dashboard`, `/admin/bookings`, `/admin/vendors`, `/admin/disputes`, `/admin/reviews`, `/admin/clients`, `/admin/financials`, `/admin/audit`, `/admin/trust`, `/admin/directory`, `/admin/commerce`

---

7. SQL migrations (`backend/sql/`)

Run in **numeric order** in Supabase. Key migrations:

File	Topic
`001_users_and_vendor_onboarding.sql`	Core users + vendors
`010_booking_requests.sql`	Booking requests table
`014_chat_conversations_messages.sql`	Chat
`017_booking_reviews.sql`	Reviews
`018_admin_console.sql`	Admin
`021_dispute_one_active_per_booking.sql`	Disputes
`026_stripe_connect_payments.sql`	Stripe + payment_status
`033_booking_daily_capacity.sql`	maxBookingsPerDay trigger
`035_client_preferred_name.sql`	Client onboarding
`036_booking_cancel_completion.sql`	Cancel audit + auto-release

---

8. Week of deep reading (feature by feature)

Goal: after one week you can explain any user action from **button click → API → DB**, starting with Auth, then client journey, then vendor journey, then money/chat/admin.

Work in the repo with the app running (`make start` backend + frontend). Prefer reading real files over memorising this PDF.

Daily ritual (45–90 min per feature slice)

1. Open the **URL** in the browser and click the primary action.

2. DevTools → Network: note method + path + request body.

3. Open the matching `lib/*Api.ts` function.

4. Jump to backend `features//http/.py` for that path.

5. Follow one call into business logic + contract.

6. Write 5–10 lines in your notes: sequence + one “aha” + one question.

7. Tick the day checklist below.

We will add full Walkthrough subsections into §6 as you finish each day (Auth is done; Bookings next, etc.).

Day 1 — Skeleton + Trace method + Auth UI→API

Morning (map)

□ Read §§1–5 of this guide
□ Read `cursor.md` “Repository layout” + golden rules
□ Open `backend/app/main.py` → `backend/app/http/router.py` (see `/api/v1`)
□ Open `frontend/app/layout.tsx` → `AuthProvider`

Afternoon (Auth walkthrough §6.1)

□ `/register` form → `useUnifiedRegister` → `lib/auth-api.ts` `signUp`
□ Backend `POST /api/v1/auth/signup` in `features/auth/http/routes.py`
□ Cookies in `session.py` + names in `core/constants.py`
□ `/login` same path via `signin`
□ Visit `/client/dashboard` logged out → `RequireAuth` redirect

Done when: you can narrate register + login without looking at this doc.

Day 2 — Session hardening + portals + roles

□ `GET /api/v1/auth/me` + `hydrate_user_from_db` (`accounts.py`)
□ `lib/auth-interceptors.ts` — force a 401 path (expired access) mentally
□ Client layout vs vendor layout vs admin layout guards

☐ `authRouting.ts` — `resolvePostAuthPath` + `?next=`
☐ Admin login vs unified login (`useAdminLogin`)
☐ Optional: local auth mode flag in settings / `local_store.py`

Done when: you know why cookies + `/me` beat storing JWTs in JS, and how role gates work on both sides.

Day 3 — Marketplace browse → vendor detail → request booking (start)

☐ `/client/browse` → `MarketplaceExploreView` → `lib/clientExploreApi.ts`
☐ Backend `features/vendors/http/explore.py` + `search.py` (limit/offset)
☐ Vendor detail page → `VendorBrowseDetailBody` / lightbox
☐ Open booking modal → note which API will create the booking (`clientBookingsApi` / create endpoint)
☐ Skim §6.2 client bookings map

Done when: you can point to the browse search request and the booking-create request in Network.

Day 4 — Client booking vertical slice (full Trace)

☐ `app/client/(portal)/bookings/page.tsx` → `ClientBookingsView`
☐ `lib/clientBookingsApi.ts` — list + detail paths
☐ `features/bookings/http/client.py` — match each UI action to a route
☐ `status.py` + `create.py` + `contracts/booking.py`
☐ Status changes: accept quote / cancel (read handlers only)

Done when: you can draw client booking list → detail → one status change end to end.

Day 5 — Pay + complete (Stripe)

☐ Checkout button → `lib/bookingCheckoutApi.ts`
☐ Backend checkout in `bookings/payments.py` + Stripe helpers
☐ Webhook entry `features/.../stripe` / router webhook path
☐ Confirm completion UI → matching POST
☐ Skim `jobs/run_booking_maintenance.py` (48h auto-release idea)

Done when: you know who creates the Checkout session and what confirms payment.

Day 6 — Vendor side + onboarding + Connect

□ Vendor bookings UI + `vendorBookingsApi.ts` + `bookings/http/vendor.py`
□ Accept/decline + quote path
□ `components/vendor-onboarding/` — how profile payload is saved
□ Payments / Stripe Connect onboarding (`vendorPaymentsApi` + backend)
□ `RequireVendorApproved` behaviour

Done when: you can contrast client vs vendor booking endpoints for the same booking id.

Day 7 — Chat, notifications, disputes, admin (survey) + recap

□ Chat portal → `chatApi.ts` → `features/chat/`
□ Notification bell → notification APIs + `notifications/copy.py`
□ Dispute “report a problem” → dispute routes
□ Admin dashboard charts (Recharts) → `adminPlatformApi` one endpoint
□ Recap: rewrite the Auth diagram from memory; list every feature you still can’t Trace

Done when: you have a personal backlog of “Walkthroughs to add to §6” for week two.

Progress log (fill in as you go)

Day	Feature sliced	Walkthrough added to §6?	Open questions
1	Auth UI + signup/signin	Yes (§6.1)	
2	Session + guards		
3	Browse → book start		
4	Client bookings		
5	Payments		
6	Vendor + Connect		
7	Chat / disputes / admin		

When you finish a day, ask in chat to **expand that feature's §6 section into a full Walkthrough** (same depth as Auth). That is how this PDF becomes a complete product map.

---

9. Quick file-finder cheat sheet

I want to understand…	Start here
All API routes	`backend/app/http/router.py`
DB access	`backend/app/core/db.py` + feature `db.py` / `queries.py`
Env vars	`backend/app/core/config.py` + `backend/.env.example`
DB schema history	`backend/sql/*.sql`
All frontend pages	`frontend/app/**/page.tsx`
Shared buttons/modals	`frontend/components/ui/`
Client vs vendor nav	`frontend/components/portal-shell/portalNav.ts`
Booking copy/strings	`frontend/features/bookings/bookingConfirmCopy.ts`
Notification text	`backend/app/features/notifications/copy.py`
Booking maintenance cron	`backend/app/jobs/run_booking_maintenance.py`

Generated for Eventtz. Keep `cursor.md` updated when architecture changes.